AN ACADEMIC INTERNSHIP REPORT

on

Job Assistant

An AI Helper for Student Job Applications
using LLMs, RAG and Agents

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Internship carried out through the online course

AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents

by Ed Donner, on Udemy (July 2026)

Submitted by

Name	**Madhurjya Deep Chakravarty**
Roll No.	2481102418
Semester	5th
Batch	2024-28

Department of Computer Science and Engineering

Jorhat Engineering College

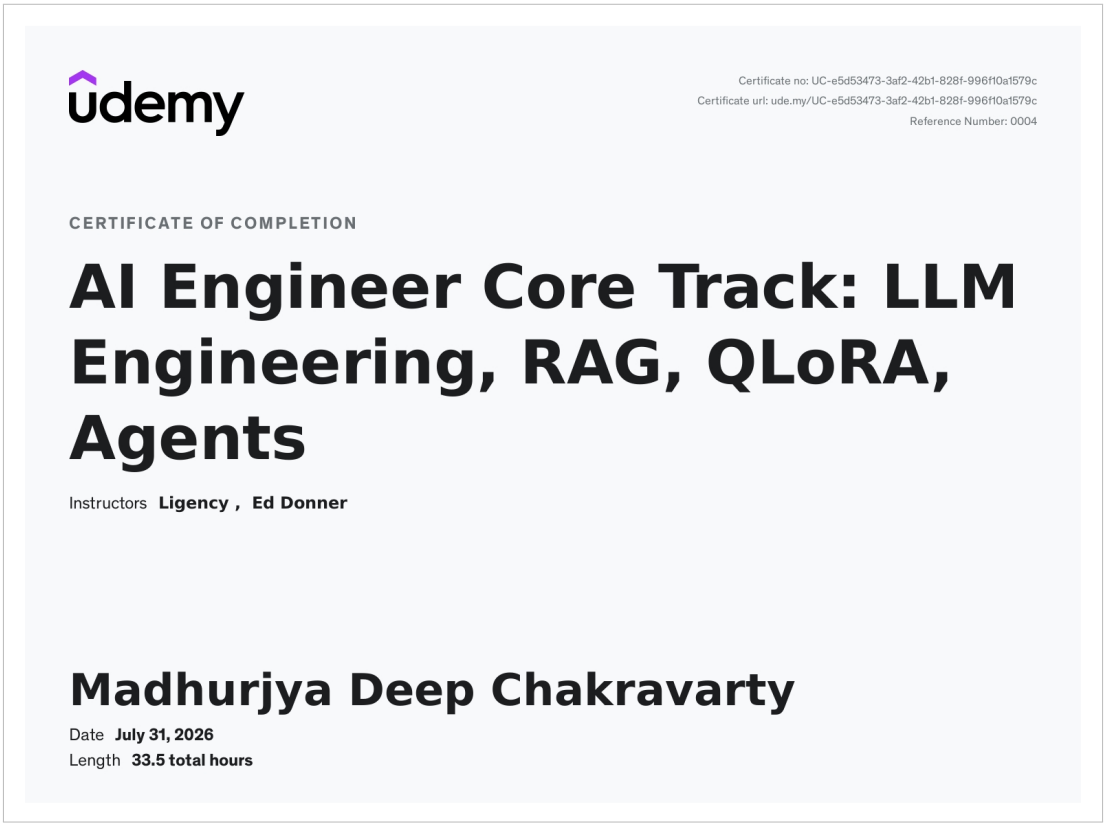
Jorhat, Assam 785007

(Affiliated to Assam Science and Technology University)

Academic Session 2026-27

Course Completion Certificate

I finished the course on **31 July 2026**. The certificate given by Udemy is shown below.



Certificate No.	UC-e5d53473-3af2-42b1-828f-996f10a1579c
Verify at	`https://ude.my/UC-e5d53473-3af2-42b1-828f-996f10a1579c`
Course length	33.5 total hours
Instructor	Ed Donner
Date of completion	31 July 2026

Declaration

I, **Madhurjya Deep Chakravarty**, a 5th semester student of B.Tech in Computer Science and Engineering (Batch 2024-28) at Jorhat Engineering College, Roll No. **2481102418**, declare that this academic internship report titled **"Job Assistant: An AI Helper for Student Job Applications using LLMs, RAG and Agents"** is a record of my own work. The internship was done in July 2026 through the online course *"AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents"* on Udemy.

The project described here was planned and built by me. This report has not been submitted to any other university or institution for any degree or diploma. Wherever I have used the work of others, I have mentioned it in the references.

Place: Jorhat
Date: _____

_____
(Madhurjya Deep Chakravarty)
Roll No.: 2481102418

Acknowledgement

I want to thank everyone who helped me during this internship.

First, I thank the **Department of Computer Science and Engineering, Jorhat Engineering College**, for making the academic internship a part of our course and for letting us choose an area we really want to learn.

I am very thankful to **Mr. Ed Donner**, the instructor of *"AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents"*. He explains hard topics in a simple way, and every week has a small project, so I was always building something and not just watching videos. Most of what I know about working with LLMs today started from his course.

I also thank the people who make and maintain the free tools I used, like FastAPI, React, ChromaDB and Tailwind CSS, and the teams behind the Gemini and Firecrawl APIs. Their documentation helped me a lot when I got stuck.

Lastly, I thank my parents, teachers and friends for their support and patience during this time.

Madhurjya Deep Chakravarty
Roll No.: 2481102418

Abstract

This report covers my academic internship, done in July 2026 through the 33.5-hour Udemy course *"AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents"* by Ed Donner. The first part of the report is about the course. It taught how to use Large Language Models (LLMs) through their APIs, how to run and compare open-source models, how to build Retrieval-Augmented Generation (RAG) systems with a vector database, how to fine-tune models with LoRA and QLoRA, and how to build agents that can call tools by themselves.

The second part is about **Job Assistant**, a web application I built using these ideas. Many students apply to internships with the same resume everywhere, and some postings are not even safe. In Job Assistant, a student first makes a profile from their resume PDF or LinkedIn PDF. Then for any job link or job text, the app gives a report: should they apply, how well they fit, which skills are missing, and any red flags like a training fee or a service bond. For a saved job, it can write a tailored one-page resume as a LaTeX file (ready for Overleaf) and a short cover letter. A tracker keeps all jobs in one place, and a chat assistant can read job links and search the profile.

The backend uses Python and FastAPI, the frontend uses React and Tailwind CSS, and Google Gemini is the language model. A knowledge hub of about 40 notes on resumes and hiring is searched with ChromaDB before the model writes anything. Plain Python checks are run after the model, so that the app does not invent skills that the student does not have.

Keywords: Large Language Models, RAG, Vector Database, AI Agents, FastAPI, React, Resume Tailoring.

Contents

Course Completion Certificate . . . . . **ii**
Declaration . . . . . **iii**
Acknowledgement . . . . . **iv**
Abstract . . . . . **v**
1 Introduction **1**
1.1 About the Internship 1
1.2 The Problem I Wanted to Solve 1
1.3 Objectives 1
2 What I Learnt in the Course **2**
2.1 Structure of the Course 2
2.2 Weeks 1 and 2: Talking to LLMs through Code 2
2.3 Weeks 3 and 4: Open-Source Models and Picking the Right One 3
2.4 Week 5: Retrieval-Augmented Generation (RAG) 3
2.5 Weeks 6 and 7: Fine-Tuning 4
2.6 Week 8: Agents 4
2.7 Tools Used in the Course 4
2.8 From the Course to my Project 5
2.9 Main Takeaways 5
3 Project Design **6**
3.1 Main Features 6
3.2 Architecture 6
3.3 API Design 7
4 How the Project Works **8**
4.1 Login and the Profile 8
4.2 The Knowledge Hub (the RAG Part) 8
4.3 Checking a Job 8
4.4 Tailored Resume and Cover Letter 9
4.5 Tracker and Chat Agent 10
5 Testing and Results **11**
5.1 A Full Run with Sample Data 11
5.2 Test Cases 11
6 Conclusion and Future Scope **12**
6.1 What I Learnt 12
6.2 Limitations 12
6.3 Conclusion 12
6.4 Future Scope 12
References **13**

List of Figures

2.1 How RAG works, in short . . . 3
2.2 The agent loop . . . 4

3.1 Architecture of Job Assistant . . . 7
3.2 FastAPI's automatic test page . . . 7

4.1 Profile page . . . 8
4.2 Full job report . . . 9
4.3 Checking a job . . . 9
4.4 Steps for writing a tailored resume . . . 10
4.5 Tailored resume and cover letter for a saved job . . . 10
4.6 Login, tracker and chat assistant . . . 10

List of Tables

1.1 Internship details . . . 1

2.1 Week-wise summary of the course . . . 2
2.2 Main tools and libraries used in the course . . . 5
2.3 Course ideas used in the project . . . 5

3.1 Technology stack . . . 6
3.2 Main API routes . . . 7

5.1 Job reports for the five sample postings . . . 11
5.2 Test cases . . . 11

Chapter 1

Introduction

1.1 About the Internship

As a part of the B.Tech curriculum, every student has to complete an academic internship. I did mine in July 2026 through a self-paced online course on Udemy. I picked this course because almost every software team now uses LLMs in some way, and I had only used ChatGPT as a user. I wanted to learn how they are used inside real software. The details are in Table 1.1.

Table 1.1: Internship details

Item	Details
Type	Academic internship (online course plus own project)
Course	AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents
Platform and instructor	Udemy, Ed Donner
Length and period	33.5 hours of video with weekly projects; July 2026

1.2 The Problem I Wanted to Solve

During placement season, I saw my seniors and friends struggle with the same things. They send one resume to every company, are not sure if they are even eligible ("2026 graduates only"), write every cover letter from scratch and lose track of where they applied. Some postings are not even safe: they ask for a "training fee" or bank details on WhatsApp, and a first-time job seeker may not know these are warning signs.

An LLM can help with all of this, but only if it is used carefully. Ask a chatbot to "improve my resume" and it often adds skills you do not have, which can get you into trouble in an interview. So my goal was to build something useful that also stays honest.

1.3 Objectives

1. To learn the basics of LLM engineering: model APIs, prompting, open-source models, RAG, fine-tuning and agents.
2. To build a web app that reads a student's resume into a clean profile, and checks any job posting against it, including red flags.
3. To write a tailored resume and cover letter using only true facts, guided by a knowledge hub of hiring advice (RAG).
4. To keep each user's data private with a proper login.

Chapter 2

What I Learnt in the Course

2.1 Structure of the Course

The course runs as an eight-week programme. Each week has a few hours of video and ends with a working project, usually in a Jupyter notebook. Table 2.1 gives a short summary of each week.

Table 2.1: Week-wise summary of the course

Week	Topics	Project
1	How LLMs work, tokens, context window, calling a frontier model through its API, system and user prompts, running models locally with Ollama	Website summariser and a company brochure maker
2	APIs of OpenAI, Anthropic and Google; building a quick UI with Gradio; chat with memory; tool calling	Customer support assistant for an airline
3	Open-source models on Hugging Face: pipelines, tokenisers, quantisation, using a free GPU on Google Colab	Meeting minutes from an audio recording
4	Choosing a model: benchmarks, leaderboards, cost and speed; LLMs for writing code	Python to C++ code converter
5	RAG: embeddings, chunking, vector stores like Chroma, LangChain	Q&A assistant over a company's documents
6	Fine-tuning a frontier model: collecting and cleaning data, a simple baseline, measuring results	Product price predictor
7	Fine-tuning an open-source model with LoRA and QLoRA, training settings, tracking runs	QLoRA price predictor
8	Agents: tools, planning, many agents working together, deployment	Multi-agent deal finder (capstone)

2.2 Weeks 1 and 2: Talking to LLMs through Code

The first thing I learnt is that an LLM does not read words. It reads *tokens*, which are small pieces of words. A model can only look at a fixed number of tokens at once (its *context window*), and APIs charge money per token. So shorter prompts are cheaper and often better too.

Every API call has two kinds of messages. The *system prompt* gives the model its fixed rules ("you are a helpful assistant, never do X"). The *user prompt* is the actual task. Ed Donner showed how much the output changes just by changing the system prompt, and this is something I used a lot later. Another useful trick was asking the model to reply in *JSON*, a fixed text format that a program can read directly. Without it, you have to guess where the

answer starts and ends in a paragraph.

In week 2 we compared the APIs of OpenAI, Anthropic (Claude) and Google (Gemini). They look different, but the idea is the same: a list of messages goes in, and a reply comes out. A chat "memory" is nothing magic either. The program simply sends the whole conversation again with every new message. We used Gradio to make a chat window in a few lines of Python, and we built an airline support assistant that could also make images and speak its answers. This is also where we first saw *tool calling*: the model can say "please run this function for me" instead of answering straight away.

2.3 Weeks 3 and 4: Open-Source Models and Picking the Right One

Week 3 was about open-source models on Hugging Face. I learnt what a tokeniser is in practice, how *quantisation* (storing the model's numbers with fewer bits) lets a big model fit on a small GPU, and how to run models on Google Colab. Week 4 answered a question I always had: which model should I use? The answer was "it depends". You look at leaderboards and benchmarks, but also at cost, speed and context length for your own task. In my project I followed this idea. I use a small, cheap Gemini model for quick jobs and a stronger one only for resume writing, where quality matters most.

2.4 Week 5: Retrieval-Augmented Generation (RAG)

This week was the most useful for my project. An *embedding* is a list of numbers that stands for the meaning of a text. Two texts with similar meaning get similar numbers, even when they share no words. For example, "built REST APIs with FastAPI" is close to "backend web development". A *vector database* like Chroma stores these numbers and can quickly find the texts closest to a question.

RAG puts these together (Figure 2.1). Before asking the model, we *retrieve* the most relevant pieces of our own documents and add them to the prompt. The model then *generates* an answer from those facts instead of from memory. This cuts down *hallucination*, which is when a model confidently says something false. It also lets a general model work with private data without any retraining.

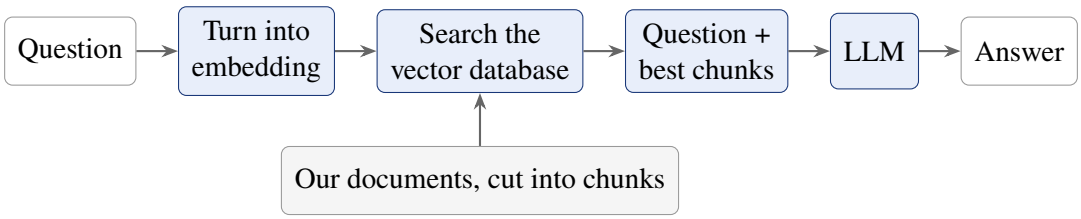


Figure 2.1: How RAG works, in short

A few practical lessons from this week stayed with me: how you cut documents into *chunks* matters a lot, each chunk should make sense on its own, and you should test whether

search is actually finding the right chunks before blaming the model.

2.5 Weeks 6 and 7: Fine-Tuning

Fine-tuning means training an existing model a little more on examples of your own task. In week 6 we built a product price predictor. Before touching any LLM, Ed Donner made us build simple baselines (even a random guess and a basic machine learning model) so that we could tell if the LLM was really better. That habit of measuring first is something I now try to follow.

Fully training a big model changes billions of numbers and needs costly hardware. *LoRA* keeps the original model frozen and trains only small extra pieces on top. *QLoRA* goes one step further and also stores the frozen model in 4-bit form, so an 8-billion-parameter model can be fine-tuned on one free Colab GPU. The main idea I took away: RAG is better for giving a model new *knowledge*, and fine-tuning is better for changing its *style or behaviour*. For my project, RAG was the right choice, because hiring advice changes and I wanted to edit it as plain text.

2.6 Week 8: Agents

An *agent* is a loop (Figure 2.2). We give the model a list of tools, which are normal functions described by a name, a purpose and their inputs. The model decides whether it needs a tool. If it does, our code runs that function and sends the result back. This repeats until the model has enough to answer.

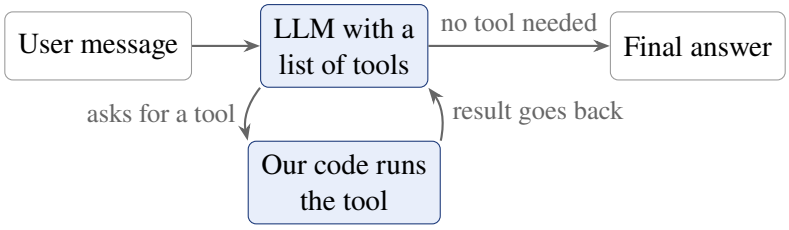


Figure 2.2: The agent loop

The capstone project was a small team of agents that worked together: one scanned deal websites, a few others estimated the right price of each product (including the fine-tuned model from week 7, running in the cloud on Modal), and one sent a phone notification when a deal looked good. It was the first time I saw all the earlier weeks come together in one system. I also learnt a safety rule here that I used directly: an agent should not be allowed to change important data without a human saying yes.

2.7 Tools Used in the Course

Apart from the ideas, the course made me comfortable with a set of tools that I had never used before (Table 2.2).

Table 2.2: Main tools and libraries used in the course

Tool	What it was used for
Jupyter notebooks, Google Colab	Running the weekly labs; Colab for free GPUs
OpenAI, Anthropic and Gemini SDKs	Calling frontier models from Python
Ollama	Running open-source models on my own laptop
Gradio	Making quick chat and demo interfaces
Hugging Face Transformers	Loading open-source models, tokenisers and pipelines
LangChain, Chroma	Building the RAG pipeline and vector store
PEFT, bitsandbytes, Weights & Biases	LoRA/QLoRA fine-tuning, 4-bit models, tracking training runs
Modal	Running a model in the cloud for the capstone

2.8 From the Course to my Project

Table 2.3 shows where each idea from the course shows up in Job Assistant.

Table 2.3: Course ideas used in the project

Course idea (week)	Where it is used in Job Assistant
APIs, system prompts (1, 2)	One helper function talks to Gemini; each feature has its own rules as a system prompt
JSON output (1, 2)	Profile, job report and resume come back in a fixed JSON shape that the code checks
Choosing a model (4)	A cheap, fast model for most jobs; a stronger one only for the resume; a fallback chain when a model is busy
Embeddings, vector store (5)	ChromaDB stores the knowledge hub and each user's profile chunks
RAG (5)	Job report, resume and cover letter all look up advice from the knowledge hub first
Measuring before trusting (6, 7)	A small test set checks that the knowledge search finds the right notes
Tools and agents (2, 8)	Chat assistant with five tools; saving a job needs the user's click

2.9 Main Takeaways

Four ideas from the course changed how I build things:

1. **Give the model the facts.** A model writing from memory guesses. A model given the right text (RAG) mostly stays with it.
2. **Ask for a fixed format.** JSON output turns a chatty reply into data that code can check.
3. **Measure before trusting.** A simple baseline and a small test set tell you if a change really helped.
4. **Keep a human in the loop.** The model suggests; the person decides.

Chapter 3

Project Design

3.1 Main Features

Job Assistant is a web app for students looking for software or hardware internships and jobs. It has four main parts:

1. **Profile.** A resume PDF and/or LinkedIn PDF is read into one clean profile, which the student checks and saves.
2. **Check a job.** For a job link or text, the app gives a verdict (apply, apply after fixes, maybe or skip), a fit score out of 10, the requirements the student has or misses, eligibility, red flags and next steps.
3. **Tracker.** Saved jobs move through saved, applied, interview, rejected or offer. Each has three tabs: report, tailored resume (LaTeX for Overleaf) and cover letter.
4. **Chat.** An assistant that can read job links, search the profile, list saved jobs and suggest saving one.

3.2 Architecture

Figure 3.1 shows how the parts fit together. The React **frontend** keeps the login token and talks only to my FastAPI **backend**, through a REST API that sends JSON. In the backend, every request passes a login check and then goes to one of five parts. The backend calls three **outside services** and keeps its **data** in two places. SQLite has a `users` table (email, password hash, profile) and an `applications` table (each saved job with its report, resume and letter), and every row has a `user_id` so each user only sees their own jobs. ChromaDB holds the shared knowledge hub and one small collection per user for their profile. API keys never leave the backend; the only thing the browser sends elsewhere is the finished `.tex` file, straight to Overleaf. Table 3.1 lists the tools and why I chose them.

Table 3.1: Technology stack

Part	Tool	Why
Backend	Python, FastAPI	Quick to write; checks request data; free test page
Frontend	React, Vite, Tailwind CSS	Pages from small reusable parts; fast styling
LLM	Gemini 3.5 Flash Lite, 3.8 Flash	Good free tier; JSON replies and tool calling
Vector database	ChromaDB, all-MiniLM-L6-v2	Runs on a laptop; small, free embedding model
Database	SQLite	One file, no setup, enough for this app
Reading input	Firecrawl, BeautifulSoup, pypdf	Job pages and PDFs into clean text
Login	bcrypt, JWT	Safe passwords and a signed login token
Resume output	LaTeX	One-page PDFs that hiring software can read

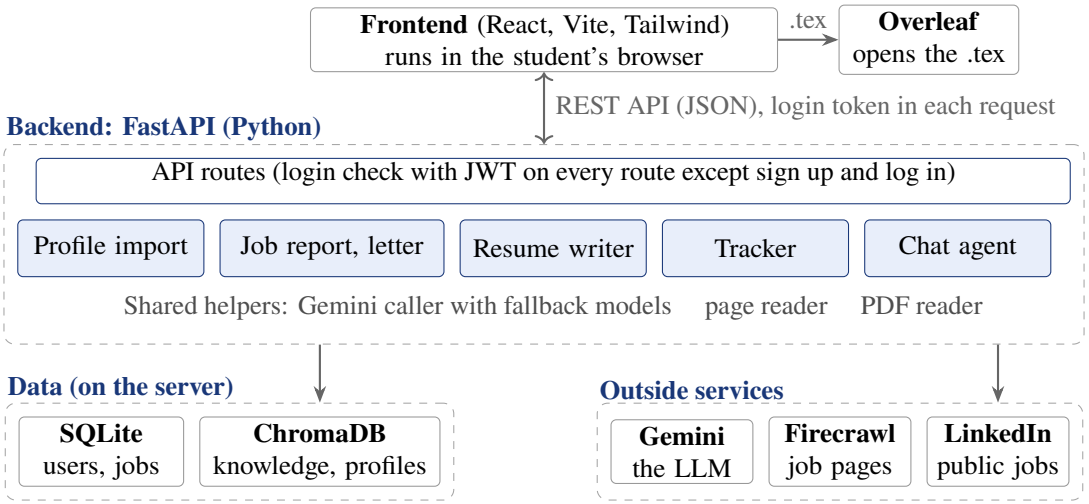


Figure 3.1: Architecture of Job Assistant

3.3 API Design

The backend is a REST API. The frontend sends a request to a URL with a method (GET to read, POST to create, PATCH to change, PUT to replace, DELETE to remove), and gets JSON back. Only sign up and log in are open. Every other route needs the login token, and FastAPI checks it before the route even runs. Table 3.2 groups the routes. Figure 3.2 (right) shows the test page FastAPI makes on its own at `/docs`, which I used a lot before the frontend was ready.

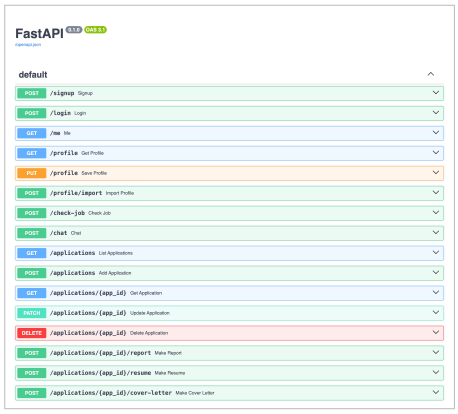


Figure 3.2: FastAPI's automatic test page

Table 3.2: Main API routes

Group	Routes	What they do
Account	POST `/signup`, POST `/login`, GET `/me`	Sign up, log in, who am I
Profile	POST `/profile/import`, GET and PUT `/profile`	Read PDFs; load or save profile
Job check	POST `/check-job`	Job report for a link or text
Tracker	GET, POST `/applications`; GET, PATCH, DELETE `/applications/{id}`	List, save, open, change, delete
Per job	POST `/applications/{id}/report`, `.../resume`, `.../cover-letter`	Make report, resume or letter
Chat	POST `/chat`	Talk to the agent

Chapter 4

How the Project Works

4.1 Login and the Profile

Passwords are stored only as a bcrypt *hash*, a one-way scramble that cannot be turned back into the password. On login, the backend gives a *JWT*, a small signed token that says "this is user 5" and expires in 7 days. Every query filters by the user's id, so someone else's job just gives "Job not found". The login page is in Figure 4.6a.

For the profile, the student uploads a resume PDF, a LinkedIn PDF, or both. The text is taken out with pypdf, and Gemini fills a fixed profile shape (Figure 4.1), keeping an entry once if it is in both PDFs. My first version stored plain resume text cleaned by the model, and once that step quietly dropped the student's *name*. With a fixed shape, code can see a missing name, and it checks the name, email and phone against the PDF text.

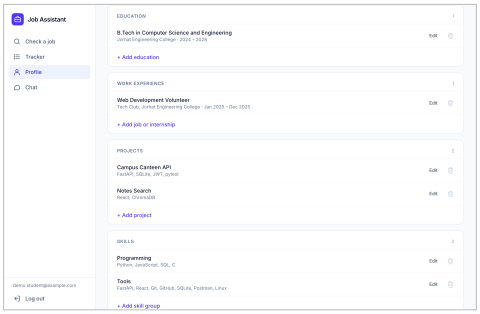


Figure 4.1: Profile page

LinkedIn links cannot be scraped (its terms forbid it), so the app reads LinkedIn's "Save to PDF" file.

4.2 The Knowledge Hub (the RAG Part)

To give good advice, the model needs good advice to read. I collected about 40 Markdown notes, mainly from a recruiter's YouTube channel (Headless Headhunter), his guide and resume template, plus research notes on hiring in 2025 and 2026 and six example resumes. They are cut at headings into 713 small chunks and stored in ChromaDB.

Two things went wrong at first. The embedding model silently ignores anything after 256 tokens, so chunks are now cut to at most 254 tokens. And searching with a whole job posting gave poor results, while four or five small questions ("red flags and scams", "hard filters") found the right advice. On 33 test questions, the right note is now always in the top five.

4.3 Checking a Job

The student gives a job link or the job text (Figure 4.3a). Firecrawl turns a link into clean text, and public LinkedIn job pages are read directly. The backend searches the knowledge hub, puts the advice, the profile and the posting into one prompt, and Gemini replies with a report in a fixed JSON shape.

Some things I do not leave to the model:

- **The score.** Asked for a number, the model gave the same job different scores on different runs. Now code works it out from the requirement list (must-haves count double).
- **Graduation year.** "2026 graduates only" against a 2028 graduate is arithmetic, which code does reliably.
- **Scam signs.** A serious red flag (a fee, bank details before an offer, chat-only hiring) or a failed condition forces the verdict to "Skip" (Figure 4.3b).

Figure 4.2 (right) shows a full report for a backend internship that fits the student.

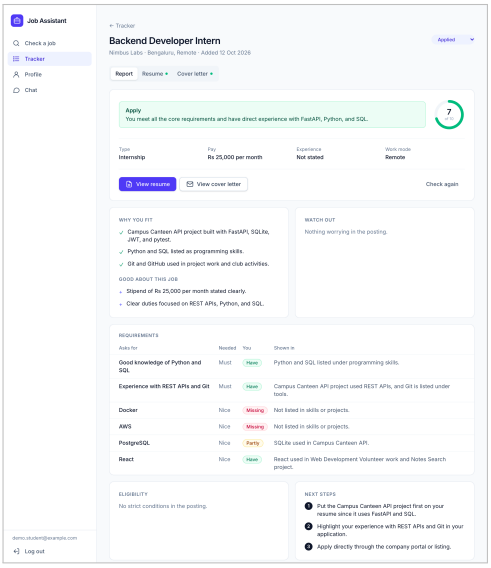
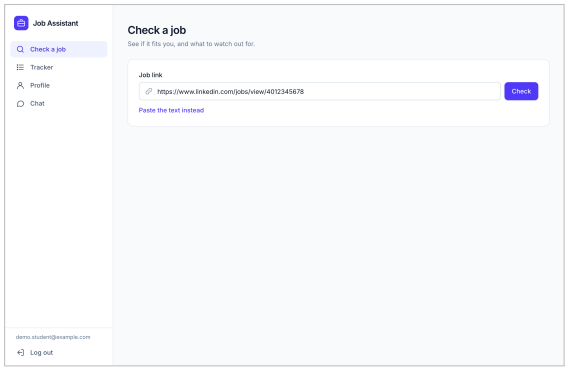
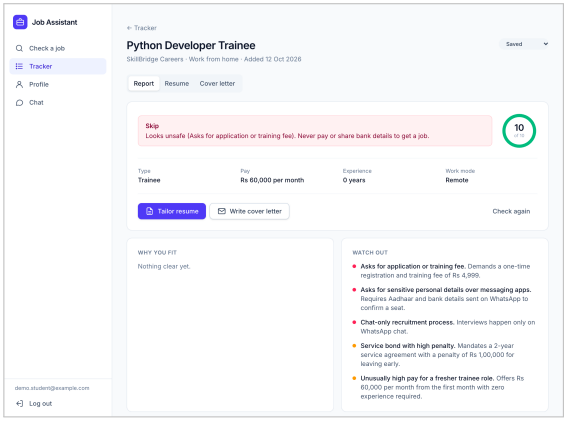


Figure 4.2: Full job report



(a) "Check a job" page



(b) Report for an unsafe posting

Figure 4.3: Checking a job

4.4 Tailored Resume and Cover Letter

The resume is made in five steps (Figure 4.4). Gemini lists what the job asks for, the knowledge hub gives fitting advice and one or two example resumes, and a stronger model (Gemini 3.8 Flash) rewrites the student's real experience for this job. Plain Python then catches anything it knows is wrong, like a skill, number or date not in the profile, and a "reviewer" pass looks for weak bullets. The writer gets one chance to fix them, then code removes anything still unsupported. The result is a LaTeX file that opens in Overleaf in one click (Figure 4.5a).

The cover letter is 120 to 200 words: one real reason for this company from the posting, one or two matching profile items, and a plain closing line. It uses only profile facts, Indian

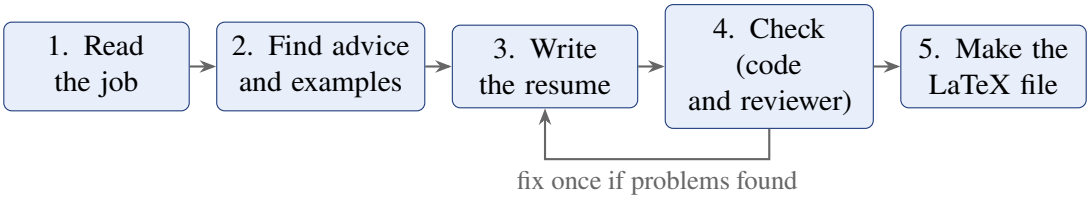
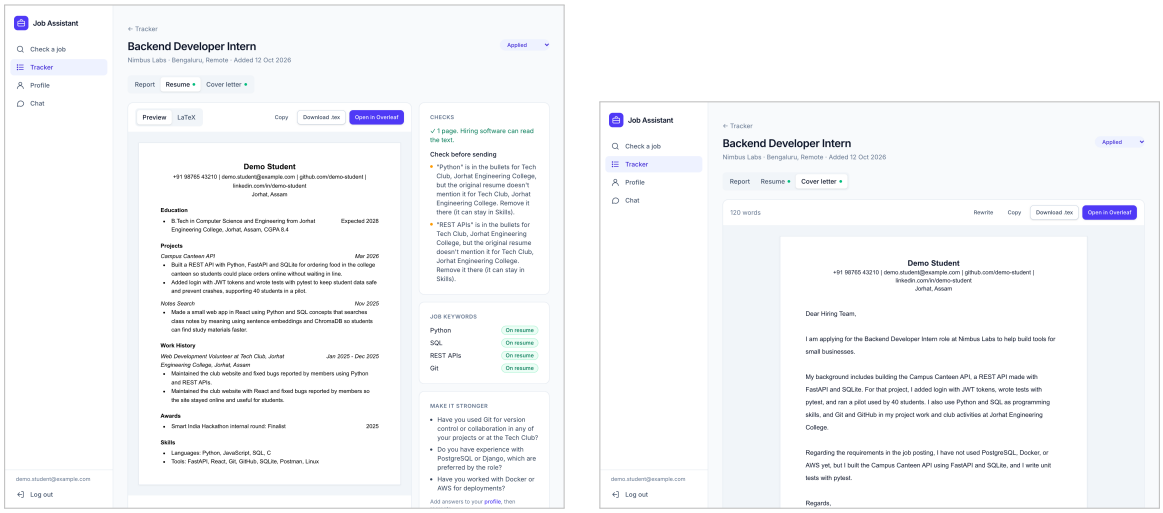


Figure 4.4: Steps for writing a tailored resume

English spelling (American for US jobs) and the same header as the resume (Figure 4.5b).

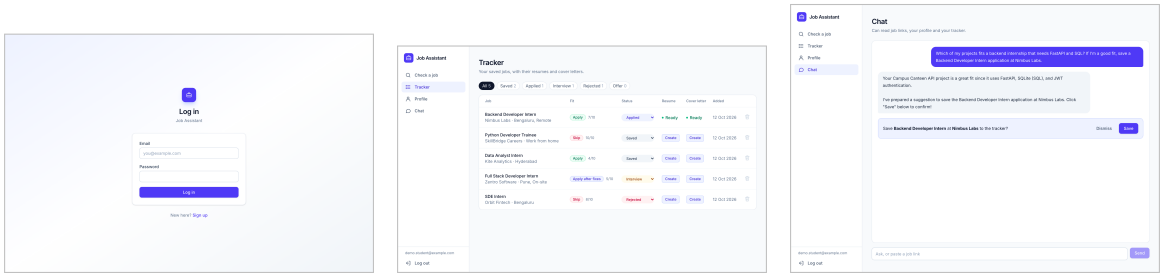


(a) Resume tab with preview and checks **(b)** Cover letter tab

Figure 4.5: Tailored resume and cover letter for a saved job

4.5 Tracker and Chat Agent

The tracker lists saved jobs with their verdict, score, status and documents (Figure 4.6b). The chat assistant (Figure 4.6c) is the agent loop from week 8. It has five tools: read a job page, read a company website, search the profile, list saved jobs and save a job. It makes at most five tool calls per message, and the save tool only makes a suggestion, so nothing is saved until the student clicks *Save*.



(a) Login **(b)** Tracker **(c)** Chat waiting for a click

Figure 4.6: Login, tracker and chat assistant

Chapter 5

Testing and Results

5.1 A Full Run with Sample Data

To test without using anyone's real data, I made a sample student ("Demo Student", a 2028 B.Tech CSE graduate with a FastAPI project and a React project) and five sample postings. Table 5.1 shows what the app said. The same jobs are in the tracker in Figure 4.6b.

Table 5.1: Job reports for the five sample postings

Posting	Verdict	Score	Reason given by the app
Nimbus Labs, Backend Intern	Apply	7	Has FastAPI, Python and SQL; Docker and AWS missing
Zentro, Full Stack Intern	Apply after fixes	5	Knows React and Git; should add Node.js and MongoDB
Kite Analytics, Data Intern	Apply	4	Meets the graduation year and core needs (see Section 6.2)
Orbit Fintech, SDE Intern	Skip	8	Wants 2026 graduates only; the student graduates in 2028
SkillBridge, Python Trainee	Skip	10	Asks for a Rs 4,999 fee and bank details on WhatsApp

5.2 Test Cases

I tested the API with `curl` (a command-line tool for sending web requests) and the screens by hand. All the cases in Table 5.2 passed.

Table 5.2: Test cases

No.	Test	Result	Status
1	Sign up with a 3-letter password	Error 400: needs at least 8 characters	Pass
2	Log in with a wrong password	Error 401	Pass
3	Open the tracker without logging in	Error 401	Pass
4	User B opens user A's saved job	Error 404, "Job not found"	Pass
5	Set a job's status to "hired"	Error 400 with the allowed list	Pass
6	Check a job with no link and no text	Error 400 with a clear message	Pass
7	Backend posting that fits	Apply, 7/10, Docker and AWS missing	Pass
8	Posting for 2026 graduates only	Skip, failed condition found by code	Pass
9	Posting that asks for a fee	Skip, three serious red flags	Pass
10	Tailored resume	One page, readable by hiring software; 2 bullets flagged	Pass
11	Ask the chat to save a job	Save and Dismiss shown; nothing saved until clicked	Pass
12	Knowledge search, 33 questions	Right note in top 5 for all 33	Pass

Chapter 6

Conclusion and Future Scope

6.1 What I Learnt

Before this internship I thought building with AI meant writing one clever prompt. I now see it as normal software engineering with one unreliable part in the middle. The model is very good at reading and writing, but it is not good at counting, comparing years or staying strictly honest. So the real work is in deciding what the model should do and what plain code should do, giving the model the right information (RAG), and checking what comes back. I also got much more comfortable with FastAPI, React, REST APIs and LaTeX.

6.2 Limitations

- The verdict and score can disagree. Kite Analytics got "Apply" with only 4 out of 10, though Excel, a must-have, was missing. A simple code rule would fix this.
- The score counts only skills, so the fee-asking posting got 10 out of 10. Its verdict (Skip) is right, but the score alone can mislead.
- The resume writer sometimes puts a tool under a job where the profile does not mention it. The checks catch it and warn the user, but it should not happen at all.

6.3 Conclusion

The course gave me a clear, hands-on base in LLM engineering, from API calls to RAG, fine-tuning and agents. Job Assistant puts most of these ideas into one working app. It turns a resume into a clean profile, gives an honest report for any job including red flags, writes a tailored resume and a cover letter from true facts only, keeps track of every application, and has a chat assistant that cannot change data without permission. It was tested with sample data, and all the main test cases passed.

6.4 Future Scope

- Put the app online so that my classmates can use it, with a proper database like PostgreSQL.
- Make the fit score include red flags and eligibility, and stop "Apply" when a must-have is missing.
- A browser extension to check a job straight from LinkedIn or Internshala.
- Use what I learnt in weeks 6 and 7 to fine-tune a small open-source model on good resume bullets, so it can run without a paid API.

References

[1] Ed Donner, *AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents*, online course on Udemy, 2026.

[2] Course code and notebooks: `https://github.com/ed-donner/llm_engineering`

[3] Headless Headhunter (recruiter), YouTube resume review videos, his written guide "How to Get a Job" and resume template, and his page of successful resumes: `https://www.youtube.com/@HeadlessHeadhunter`, `https://www.headlessheadhunter.org`. This is the main source of the knowledge hub.

[4] Other sources used for the knowledge hub's research notes: help pages of hiring software (Greenhouse, Lever, Ashby, Workday), interviewing.io, the Tech Interview Handbook, The Pragmatic Engineer newsletter, NACE, and job scam advice from the US FTC and FBI IC3.

[5] Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", 2020. `https://arxiv.org/abs/2005.11401`

[6] Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", 2021, and Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs", 2023. `https://arxiv.org`

[7] FastAPI documentation: `https://fastapi.tiangolo.com`

[8] React and Tailwind CSS documentation: `https://react.dev`, `https://tailwindcss.com`

[9] ChromaDB documentation: `https://docs.trychroma.com`

[10] Google Gemini API documentation: `https://ai.google.dev`

[11] Firecrawl documentation: `https://docs.firecrawl.dev`

[12] Overleaf LaTeX guides: `https://www.overleaf.com/learn`